

Resolução do Exame Modelo de Recurso — Bases de Dados (Modelo 1)

 **Ano Letivo:** 2025/2026 |  **Época:** Recurso (Modelo)

 **Curso:** Engenharia Informática / Segurança Informática em Redes de Computadores

 **Instituição:** P.PORTO — ESTG

 **Unidade Curricular:** Bases de Dados

1. Independência de Dados (2 val.)

? Pergunta 1: Descreva o conceito de independência de dados e a sua importância num ambiente de bases de dados. Diferencie entre independência física e independência lógica de dados, dando um exemplo prático de cada uma.

 **Resposta:** A **independência de dados** é a capacidade de alterar o esquema de uma base de dados num determinado nível de abstração da arquitetura ANSI/SPARC sem a necessidade de reestruturar os níveis superiores. Divide-se em dois tipos:

- **Independência Física de Dados:** Capacidade de alterar o armazenamento físico dos dados (como reorganizar ficheiros, criar ou eliminar índices, mudar de disco ou alterar partições) sem afetar o esquema conceptual (lógico) nem as aplicações dos utilizadores.
 - *Exemplo prático:* O DBA cria um novo índice na coluna `email` da tabela `Clientes` para acelerar pesquisas. As aplicações que fazem `SELECT * FROM Clientes WHERE email = '...'` continuam a funcionar sem qualquer alteração de código.
- **Independência Lógica de Dados:** Capacidade de alterar o esquema conceptual (como adicionar novas colunas, dividir tabelas ou renomear atributos) sem que seja necessário reescrever o código das aplicações existentes.
 - *Exemplo prático:* O DBA divide a tabela `Funcionario(codF, nome, morada, salario)` em duas tabelas: `Funcionario(codF, nome, morada)` e `Salario(codF, salario)`. Para manter as aplicações inalteradas, cria-se uma vista `CREATE VIEW Funcionario_Antigo AS SELECT ... FROM Funcionario NATURAL JOIN Salario` que simula a tabela original.

A importância da independência de dados reside na **simplificação do desenvolvimento**, no **aumento da flexibilidade evolutiva** da base de dados e na **redução significativa dos custos de manutenção** de software, uma vez que alterações físicas ou lógicas não propagam impacto às aplicações clientes.

2. 🔗 Integridade Referencial e Ações ON DELETE / ON UPDATE (2 val.)

? **Pergunta 2:** Explique o que entende por Integridade Referencial. Identifique e descreva quais as ações que se podem utilizar nas subcláusulas ON DELETE e ON UPDATE.

🔗 **Resposta:** A **integridade referencial** é uma regra de integridade do modelo relacional que garante a consistência das ligações entre tabelas, obrigando a que os valores de uma chave estrangeira (FK) na tabela filha existam previamente na chave primária (PK) da tabela pai ou sejam nulos.

Exemplo: Se a tabela `Encomendas` tiver a coluna FK `ID_Cliente` ligada à tabela `Clientes`, este valor deve obrigatoriamente existir na coluna PK `ID` de `Clientes` (ou ser NULL, caso a coluna o permita).

Para gerir a eliminação ou alteração de registos pais e evitar registos órfãos, utilizam-se as subcláusulas ON DELETE e ON UPDATE com as seguintes ações:

- **CASCADE:** Propaga a operação automaticamente para os registos filhos. Ao eliminar o registo pai, eliminam-se os filhos dependentes; ao atualizar a PK do pai, atualiza-se a FK correspondente nos filhos.
- **SET NULL:** Altera a coluna FK de todos os registos filhos correspondentes para `NULL`. Requer que a coluna FK permita valores nulos.
- **SET DEFAULT:** Altera a coluna FK de todos os registos filhos correspondentes para o valor por defeito (DEFAULT) configurado na definição da coluna.
- **NO ACTION (ou RESTRICT):** Rejeita a operação de eliminação ou atualização no registo pai se existirem registos filhos dependentes na tabela filha. É o comportamento por defeito.

3. ⚡ Triggers de Bases de Dados (2 val.)

? **Pergunta 3:** O que são triggers de bases de dados e para que servem? Quais as vantagens e desvantagens da utilização de triggers? Identifique os diferentes tipos de triggers quanto ao momento de execução.

🔗 **Resposta:** Um **trigger** (gatilho) é um bloco de código procedural armazenado no SGBD que é executado de forma automática e implícita em resposta a uma operação DML (INSERT, UPDATE ou DELETE) numa tabela.

Para que servem:

- Impor regras de negócio complexas que não podem ser expressas por restrições declarativas normais (PK, FK, CHECK).
- Criar registos de auditoria e logs de histórico de forma automática.

- Manter dados derivados ou tabelas resumo atualizadas.
- Garantir a integridade referencial complexa entre tabelas.

Vantagens:

- **Centralização da lógica:** A integridade é garantida ao nível da base de dados, protegendo os dados independentemente da aplicação cliente que acede ao sistema.
- **Automatização:** A execução é implícita e automática, eliminando código redundante nas aplicações.

Desvantagens:

- **Redução de performance (overhead):** Adiciona tempo de processamento a cada operação de escrita na tabela.
- **Efeitos ocultos e cascata:** Como a execução é implícita, pode desencadear efeitos em cascata difíceis de prever e de depurar (debug).
- **Falta de portabilidade:** A sintaxe dos triggers varia significativamente entre SGBDs (ex: Oracle PL/SQL vs SQL Server T-SQL vs MySQL).

Tipos quanto ao momento de execução:

Tipo	Quando executa	Aplicação típica
BEFORE	Antes da operação DML	Validação ou transformação de dados de entrada
AFTER	Depois da operação DML	Ações complementares, registos de auditoria
INSTEAD OF	Em vez da operação DML	Tornar vistas complexas atualizáveis

4. Subquery vs Junção (2 val.)

? Pergunta 4: Qual a diferença entre uma subquery e uma junção? Em que situações não é possível usar uma subquery? Dê um exemplo SQL que ilustre a necessidade de usar uma junção em vez de uma subquery.

 Resposta: Uma **subquery (subconsulta)** é uma instrução SELECT aninhada dentro de outra consulta externa (nas cláusulas WHERE, HAVING, FROM ou SELECT), servindo para calcular dados temporários ou escalares que alimentam a query principal.

Uma **junção (JOIN)** é uma operação que combina registos de duas ou mais tabelas na mesma linha de dados do resultado final, com base numa condição de igualdade ou lógica comum.

Quando NÃO é possível usar uma subquery: Não é possível utilizar uma subquery nas situações em que a consulta final exige a **projeção simultânea de colunas pertencentes a tabelas diferentes** no mesmo resultado. Uma subquery apenas atua como filtro ou fonte secundária de dados, limitando a projeção às colunas da tabela declarada na query externa principal.

Exemplo SQL ilustrativo: Suponha as tabelas `Aluno(codAluno, nome)` e `Inscricao(codAluno, disciplina, nota)`. Se quisermos listar o nome do aluno e a disciplina e a nota, precisamos de uma junção:

```
-- ☒ Apenas possível com JOIN (colunas de 2 tabelas no SELECT):  
SELECT A.nome, I.disciplina, I.nota  
FROM Aluno A  
INNER JOIN Inscricao I ON A.codAluno = I.codAluno;  
  
-- ☒ Uma subquery NÃO consegue projetar colunas de Inscricao E de Aluno:  
SELECT A.nome, ???  
FROM Aluno A  
WHERE A.codAluno IN (SELECT codAluno FROM Inscricao);  
-- Aqui só conseguimos ver o nome do aluno, mas não a disciplina nem a nota.
```

5. Materialização de Vistas (2 val.)

? Pergunta 5: Explique o conceito de materialização de vistas. Quais as vantagens e desvantagens desta abordagem em comparação com as vistas tradicionais? Em que contextos é recomendável a sua utilização?

 **Resposta:** A **materialização de vistas** (Materialized Views ou Indexed Views) consiste em armazenar fisicamente em disco os resultados da consulta SQL que define a vista, em vez de recalculá-la a cada vez que a vista é consultada.

Numa **vista tradicional**, apenas a definição da consulta (o SELECT) é armazenada nos metadados. Quando se consulta a vista, o SGBD executa a query subjacente em tempo real. Numa **vista materializada**, o resultado da consulta é pré-calculado e persistido como uma tabela temporária física.

Vantagens da materialização:

- **Desempenho de leitura muito superior:** Os dados já estão pré-calculados em disco; consultas complexas com agregações ou junções pesadas são devolvidas quase instantaneamente, sem recalculá-las.
- **Redução de carga no servidor:** Elimina a necessidade de executar repetidamente consultas complexas em tempo real.

Desvantagens da materialização:

- **Overhead nas operações de escrita:** O SGBD precisa de recalculer e sincronizar a vista materializada sempre que ocorrem alterações (INSERT, UPDATE, DELETE) nas tabelas base subjacentes.
- **Consumo de espaço em disco:** Os resultados ocupam armazenamento físico adicional, ao contrário das vistas tradicionais.
- **Possível desatualização dos dados:** Dependendo da política de atualização (síncrona vs assíncrona), os dados da vista materializada podem estar ligeiramente desatualizados.

Contextos recomendáveis:

- Ambientes analíticos **OLAP** com consultas complexas de leitura frequente e escrita rara.
- Relatórios periódicos com grandes agregações (SUM, AVG, COUNT) sobre milhões de registos.
- Cenários de **Data Warehousing** onde o desempenho de leitura é crítico.

6. Data Warehouses (2 val.)

? **Pergunta 6:** Descreva os principais benefícios e problemas associados aos Data Warehouses. Distinga entre um Data Warehouse e um Data Mart.

 **Resposta:** Um **Data Warehouse (DW)** é um repositório de dados histórico, integrado, orientado a assuntos e não-volátil, projetado especificamente para apoiar o processo de tomada de decisão da administração.

Benefícios:

- **Integração de dados:** Reúne dados consolidados de fontes heterogêneas (múltiplos sistemas operacionais, ficheiros, ERPs) num único local centralizado.
- **Análise histórica:** Permite avaliar tendências a longo prazo através de dados variáveis no tempo, dando suporte à tomada de decisão baseada em dados.
- **Isolamento de performance:** Evita que consultas analíticas complexas e lentas (OLAP) degradem o desempenho dos sistemas transacionais operacionais do dia a dia (OLTP).

Problemas:

- **Custo e tempo elevados:** Projetos de longa duração com custos de implementação, hardware e licenciamento muito elevados.
- **Complexidade de ETL:** O processo de Extração, Transformação e Carregamento é complexo e propenso a erros na garantia da qualidade de dados.
- **Manutenção contínua:** Dificuldade em manter o DW atualizado quando os sistemas operacionais fonte sofrem alterações estruturais.

Distinção entre Data Warehouse e Data Mart: Um **Data Mart** é um subconjunto de um Data Warehouse que suporta os requisitos de um determinado departamento ou função de negócio (ex: Marketing, Vendas, Finanças). Apresenta a vantagem de ser mais simples, mais rápido e mais barato de implementar do que um DW completo, sendo focado numa única área de análise.

7. **Exercício de Normalização de Fatura (3 val.)**

 **Pergunta 7:** Normalização da fatura da TecnoShop.

 **Resposta:**

Identificação dos Atributos

Identificamos no documento os seguintes atributos:

Letra	Atributo
A	NIF_Empresa
B	Nome_Empresa
C	Morada_Empresa
D	CodPostal_Empresa
E	NumFatura
F	Data
G	Hora
H	ATCUD
I	NIF_Cliente

Letra	Atributo
J	Nome_Cliente
K	Morada_Cliente
L	CodPostal_Cliente
M	CodEnvio
N	MetodoEnvio
O	CustoEnvio
P	PrazoEstimado
Q	RefArtigo
R	Descricao_Artigo
S	Categoria
T	Quantidade
U	PrecoUnitario
V	TaxaIVA
W	Subtotal_Linha
X	Total_Fatura

Letra	Atributo
Y	MetodoPagamento
Z	Incidencia_IVA
AA	Valor_IVA

Forma Não Normalizada (FNN)

Definição: Uma tabela que contém um ou mais grupos repetidos.

Na FNN, todos os atributos residem numa única relação com dois grupos repetitivos (artigos e resumo de IVA):

```
Fatura_FNN(E, A, B, C, D, F, G, H, I, J, K, L, M, N, O, P, X, Y,
           [Q, R, S, T, U, V, W],      ← grupo repetido: artigos
           [V*, Z, AA])                ← grupo repetido: resumo IVA
```

1 Primeira Forma Normal (1FN)

Definição: Uma relação em que a intersecção entre uma linha e uma coluna contém um e um só valor (valores atómicos). Não deve conter grupos repetidos.

Achatando os grupos repetitivos, a PK passa a ser composta:

```
Fatura_1FN(NumFatura, RefArtigo, TaxaIVA_Resumo,
           NIF_Empresa, Nome_Empresa, Morada_Empresa, CodPostal_Empresa,
           Data, Hora, ATCUD, NIF_Cliente, Nome_Cliente, Morada_Cliente,
           CodPostal_Cliente, CodEnvio, MetodoEnvio, CustoEnvio, PrazoEstimado,
           Descricao_Artigo, Categoria, Quantidade, PrecoUnitario,
           TaxaIVA_Artigo, Subtotal_Linha, Total_Fatura, MetodoPagamento,
           Incidencia_IVA, Valor_IVA)
PK: (NumFatura, RefArtigo, TaxaIVA_Resumo)
```

Dependências Funcionais (DFs) verificadas:

- \$NumFatura \rightarrow NIF_Empresa, Nome_Empresa, Morada_Empresa, CodPostal_Empresa, Data, Hora, ATCUD, NIF_Cliente, Nome_Cliente, Morada_Cliente, CodPostal_Cliente, CodEnvio, MetodoEnvio, CustoEnvio, PrazoEstimado, Total_Fatura, MetodoPagamento\$
- \$NumFatura, RefArtigo \rightarrow Quantidade, Subtotal_Linha\$

- \$RefArtigo \rightarrow Descricao_Artigo, Categoria, PrecoUnitario, TaxaIVA_Artigo\$
- \$NumFatura, TaxaIVA_Resumo \rightarrow Incidencia_IVA, Valor_IVA\$
- \$NIF_Empresa \rightarrow Nome_Empresa, Morada_Empresa, CodPostal_Empresa\$
- \$NIF_Cliente \rightarrow Nome_Cliente, Morada_Cliente, CodPostal_Cliente\$
- \$CodEnvio \rightarrow MetodoEnvio, CustoEnvio, PrazoEstimado\$

2 Segunda Forma Normal (2FN)

Definição: Uma relação que está na 1FN e todos os atributos não primos dependem totalmente da chave primária (sem dependências parciais).

Decompondo as dependências parciais sobre a chave composta (NumFatura, RefArtigo, TaxaIVA_Resumo):

```
Cabecalho_2FN(NumFatura, NIF_Empresa, Nome_Empresa, Morada_Empresa,
               CodPostal_Empresa, Data, Hora, ATCUD, NIF_Cliente, Nome_Cliente,
               Morada_Cliente, CodPostal_Cliente, CodEnvio, MetodoEnvio,
               CustoEnvio, PrazoEstimado, Total_Fatura, MetodoPagamento)
    PK: NumFatura

Artigo_2FN(RefArtigo, Descricao_Artigo, Categoria, PrecoUnitario, TaxaIVA_Artigo)
    PK: RefArtigo

LinhaFatura_2FN(NumFatura, RefArtigo, Quantidade, Subtotal_Linha)
    PK: (NumFatura, RefArtigo)

ResumoIVA_2FN(NumFatura, TaxaIVA_Resumo, Incidencia_IVA, Valor_IVA)
    PK: (NumFatura, TaxaIVA_Resumo)
```

3 Terceira Forma Normal (3FN)

Definição: Uma relação que está na 2FN e na qual nenhum atributo não pertencente à chave primária depende transitivamente de qualquer chave candidata.

Dependências transitivas detetadas:

- Em Cabecalho_2FN: \$NIF_Empresa \rightarrow Nome_Empresa, Morada_Empresa, CodPostal_Empresa\$ (transitiva via NumFatura)
- Em Cabecalho_2FN: \$NIF_Cliente \rightarrow Nome_Cliente, Morada_Cliente, CodPostal_Cliente\$ (transitiva via NumFatura)
- Em Cabecalho_2FN: \$CodEnvio \rightarrow MetodoEnvio, CustoEnvio, PrazoEstimado\$ (transitiva via NumFatura)

Extraindo estas dependências, obtemos o **esquema final normalizado (3FN)**:

```
Empresa(NIF_Empresa, Nome_Empresa, Morada_Empresa, CodPostal_Empresa)
    PK: NIF_Empresa

Cliente(NIF_Cliente, Nome_Cliente, Morada_Cliente, CodPostal_Cliente)
    PK: NIF_Cliente

MetodoEnvio(CodEnvio, MetodoEnvio, CustoEnvio, PrazoEstimado)
    PK: CodEnvio

Artigo(RefArtigo, Descricao_Artigo, Categoria, PrecoUnitario, TaxaIVA)
    PK: RefArtigo

Fatura(NumFatura, Data, Hora, ATCUD, NIF_Empresa, NIF_Cliente,
    CodEnvio, Total_Fatura, MetodoPagamento)
    PK: NumFatura
    FK: NIF_Empresa → Empresa(NIF_Empresa)
    FK: NIF_Cliente → Cliente(NIF_Cliente)
    FK: CodEnvio → MetodoEnvio(CodEnvio)

LinhaFatura(NumFatura, RefArtigo, Quantidade, Subtotal_Linha)
    PK: (NumFatura, RefArtigo)
    FK: NumFatura → Fatura(NumFatura)
    FK: RefArtigo → Artigo(RefArtigo)

ResumoIVA(NumFatura, TaxaIVA, Incidencia_IVA, Valor_IVA)
    PK: (NumFatura, TaxaIVA)
    FK: NumFatura → Fatura(NumFatura)
```

8. Modelação, SQL e Álgebra Relacional (5 val.)

a) Chave primária e chaves estrangeiras da tabela Reserva (1 val.)

 **Pergunta 8a:** Identifique a PK e FKs da tabela Reserva. Justifique.

 **Resposta:**

```
Reserva(codReserva, codPass, numVoo, dataViagem, classe, preco)
    PK: codReserva
    FK: codPass → Passageiro(codPass)
```

FK: numVoo → Voo(numVoo)

Justificação: A chave primária é `codReserva` porque identifica univocamente cada reserva efetuada. As chaves estrangeiras são `codPass`, que referencia o passageiro que efetuou a reserva na tabela `Passageiro`, e `numVoo`, que referencia o voo reservado na tabela `Voo`. Estas FKs garantem a integridade referencial — não é possível criar uma reserva para um passageiro ou voo inexistente.

b) SQL: Países com mais de 5 passageiros com reservas para o Porto em 2026 (2 val.)

? **Pergunta 8b (SQL):** Quais os países que têm mais de 5 passageiros com reservas em voos para a cidade do Porto no ano de 2026?

Resposta:

Juntamos as tabelas `Passageiro`, `Reserva`, `Voo` e `Aeroporto` (esta última ligada pelo destino do voo). Filtramos pela cidade 'Porto' e pelo ano 2026 na data de viagem. Agrupamos por país e aplicamos `HAVING` com `COUNT DISTINCT` para garantir que contamos passageiros únicos:

```
SELECT P.pais, COUNT(DISTINCT P.codPass) AS TotalPassageiros
FROM Passageiro P
INNER JOIN Reserva R ON P.codPass = R.codPass
INNER JOIN Voo V ON R.numVoo = V.numVoo
INNER JOIN Aeroporto A ON V.destino = A.codIATA
WHERE A.cidade = 'Porto'
      AND R.dataViagem >= '2026-01-01'
      AND R.dataViagem <= '2026-12-31'
GROUP BY P.pais
HAVING COUNT(DISTINCT P.codPass) > 5;
```

c) Álgebra Relacional: Aeroportos que nunca foram destino de voos com reservas (2 val.)

? **Pergunta 8c (Álgebra Relacional):** Quais os aeroportos que nunca foram destino de nenhum voo com reservas?

Resposta:

Projetamos todos os códigos IATA existentes na tabela `Aeroporto`. Depois, juntamos `Voo` com `Reserva` para encontrar os voos que efetivamente possuem reservas e projetamos os códigos

de destino desses voos. Subtraímos este conjunto do total de aeroportos para obter os que nunca foram destino:

$\text{\$TodosAeroportos} \setminus \pi_{\text{\{codIATA\}}}(\text{\text{Aeroporto}})$

$\text{\$VoosComReserva} \setminus \pi_{\text{\{bowtie_{\text{\{Voo.numVoo = Reserva.numVoo\}}}\}}}(\text{\text{Reserva}})$

$\text{\$DestinosComReserva} \setminus \pi_{\text{\{destino\}}}(\text{\text{VoosComReserva}})$

$\text{\$Resultado} \setminus \pi_{\text{\{}}(\text{\text{TodosAeroportos}} - \text{\text{DestinosComReserva}})$